

Using DTrace with MySQL

As mentioned earlier, there are two ways in which DTrace can be used with any user-level process—USDT and the PID provider. We shall see demonstrations of both these mechanisms as we start using DTrace with the MySQL server, or specifically ‘mysqld’. One thing to note here is that DTrace one-liners can be used to demonstrate a lot of what we will be doing. But, to make the learning easier, we will use D-scripts, however small they may be. Familiarity with the MySQL source code is required to derive the maximum advantage from the rest of the article.

Before I start off with writing D scripts, here are some common points worth noting:

- Like a shell script, you can make a D script executable by using `chmod +x` and specifying the location of the script interpreter using `a #!`, like: `#!/usr/sbin/dtrace`.
- You can specify various switches to *dtrace*. For example, to specify a D script to *dtrace*, you will use the `-s` switch.
- The parameters of the function being traced are available to a D script using built-in variables: `arg0`, `arg1`, `arg2`... Other built-in variables like `timestamp`, `walltimestamp` are described at docs.sun.com/app/docs/doc/817-6223/chp-variables-5?a=view.timestamp gives the current value of a nanosecond counter, which increments from an arbitrary time in the past and is useful for relative time calculation. `walltimestamp`, the current number of nanoseconds since UTC, is more useful when date/time value is required.
- `copyinstr` is used to copy the value of a `char*` type parameter to a variable in your D script. By default, strings up to a maximum size of 256 can be stored. You can change it using `#pragma D option strsize=1024`.
- When monitoring probes for a multi-threaded application, such as *mysqld*, it is essential that each thread (and its variables) is treated as such. Thread-local variables, denoted using a `self->`, makes it possible to prevent corruption of the variables of one thread by another.
- DTrace allows use of *clause-local* variables. To declare a variable as *clause-local*, specify it as `this->`, such as `this->bar`. As suggested by its name, the scope of a *clause-local* variable is limited to the probe clause or predicate, in which it is used. For more information on thread-local and clause-local variables, please refer to wikis.sun.com/display/DTrace/Variables.
- A number of macro variables are available in DTrace. A very commonly used one is `$target`: which is used in scripts using the PID provider.

Using the PID provider

To use the PID provider, you need to have a *mysqld* instance running on an (Open)Solaris system. (You won't need any special build of MySQL for this.) Please note that the function names are garbled in a binary. Hence any command, for example, *mysql_parse* will not be exactly

the same, but will have other text at the starting and the ending. We can use “nm” to see the garbled names:

```
nm mysqld | grep mysql_parse
2:1134 | 136472368| 640|FUNC|GLOB|0 |13 |_lmysql_
parse6f|pnDTHD_pkcp3_v_
```

Hence, we shall simply use the regex “*” at the beginning and end of the function name in our D scripts.

• Watching queries

```
#!/usr/sbin/dtrace -q
pid$target: "mysql_parse" entry /* This probe is fired when the execution
enters mysql_parse */
{
    printf("Query: %s\n", copyinstr(arg1));
}
```

Save this script to a file, say *watch.d*. A D script is specified to *dtrace* with the `-s` switch. The Process ID (PID) specified via the switch `-p` is automatically made available to the `$target` macro in the D script.

Now, run the D script, *watch.d*:

```
$ ptxexec dtrace -s watch.d -p 'pgrep -x mysqld'
```

Fire up a MySQL client and run some queries. The D script should display the queries that you executed from the client:

```
Query: show databases
Query: show variables
Query: show engines
Query: SELECT DATABASE()
Query: show databases
Query: show tables
Query: show tables
Query: select * from foo
```

• **Follow the query execution:** An SQL query before execution passes through various other stages, the first of which is the query parsing. The query parsing is taken care of by the *mysql_parse* function in *sql/sql_parse.cc*. Since all the other stages, such as query optimisation and, finally, execution follows from there, by using the following script, we can track all the functions that call and return from after *mysql_parse*:

```
#!/usr/sbin/dtrace
#pragma D option flowindent

pid$target: "mysql_parse" entry
{
    self->start = vtimestamp;
```